Reviewer Pack — Coxeter-Dynkin Diagrams and Monomial Ideal Complexity

Entry 7cc4fa3cad8b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 06:14:37 UTC

Coxeter-Dynkin Diagrams and Monomial Ideal Complexity

Entry ID: `7cc4fa3cad8b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 06:14:37 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Algebra (Coxeter Groups) **Field B** (complexity object): Complexity Theory: Monomial Ideal Complexity

Statement:

> ['For any given Boolean function f with n variables, the number of distinct monomial ideals generated by the set of all minterms of f is upper bounded by the number of vertices in the Dynkin diagram associated with the Coxeter group corresponding to the maximum number of distinct minimal nonzero coefficients of f.', 'Furthermore, there exists a constructive mapping from a Boolean function f to its associated Dynkin diagram that can be computed in polynomial time.']

Rationale (proposer's reasoning):

> ['The Coxeter-Dynkin diagrams provide a rich structure that encodes information about the symmetry and algebraic properties of Coxeter groups. This conjecture suggests that this structural information could be used to bound the complexity of certain Boolean functions, specifically through their monomial ideal complexity.', 'Previous work has shown connections between Coxeter groups and various aspects of algorithm design. By leveraging this connection, we may uncover new insights into the computational complexity of Boolean functions.']

Taxonomy category: `CONJUGATE_CATEGORIES` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `4bb1333246d7f603`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Spearman rank correlation coefficient between the number of distinct monomial ideals and the number of vertices in the Dynkin diagram across 30 random seeds is ≥ 0.7.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Coxeter-Dynkin diagrams" AND "monomial ideal complexity" - "Boolean functions" AND "minimal nonzero coefficients" AND "Dynkin diagram" - "polynomial time algorithm" AND "mapping" AND "Boolean function" AND "Coxeter group"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def minterms(f, n):
        terms = []
        for i in range(2**n):
            if f[i] == 1:
                term = [int(x) for x in format(i, f'0{n}b')]
                terms.append(term)
        return terms

    def monomial_ideals(mints):
        ideals = set()
        n = len(mints[0])
        for mint in mints:
            ideal = tuple(sorted([i for i, bit in enumerate(mint) if bit == 1]))
            ideals.add(ideal)
        return ideals

    def dynkin_diagram(n):
        if n == 2:  # A_1
            return {'vertices': [0], 'edges': []}
        elif n == 3:  # A_2
```

```
            return {'vertices': [0, 1], 'edges': [(0, 1)]}
        elif n == 4:  # A_3
            return {'vertices': [0, 1, 2], 'edges': [(0, 1), (1, 2)]}
        elif n == 5:  # A_4
            return {'vertices': [0, 1, 2, 3], 'edges': [(0, 1), (1, 2), (2, 3)]}
        else:
            raise ValueError("Unsupported number of variables for Dynkin diagram")

    def spearman_rank_correlation(x, y):
        n = len(x)
        x_ranks = {x[i]: i + 1 for i in range(n)}
        y_ranks = {y[i]: i + 1 for i in range(n)}

        sum_differences_squared = sum((x_ranks[x[i]] - y_ranks[y[i]]) ** 2 for i in range(n))
        rho_numerator = n * sum_differences_squared
        rho_denominator = (n * (n**2 - 1)) * (1 - (6 * sum_differences_squared) / (n**3 - n))

        return 1 - (rho_numerator / rho_denominator)

    n_values = [5, 10, 15, 20, 30, 40]
    x_values = []
    y_values = []

    for n in n_values:
        f = generate_boolean_function(n)
        mints = minterms(f, n)
        ideals = monomial_ideals(mints)
        dynkin = dynkin_diagram(n)

        x_values.append(len(ideals))
        y_values.append(len(dynkin['vertices']))

    rho = spearman_rank_correlation(x_values, y_values)

    return {
        "metric_name": "Spearman rank correlation",
        "metric_value": rho,
        "instances_tested": len(n_values),
        "conjecture_holds": rho >= 0.7,
        "counterexample": "" if rho >= 0.7 else f"Spearman rank correlation < 0.7: {rho}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_rho = sum(r['metric_value'] for r in results) / len(results)
    std_rho = math.sqrt(sum((r['metric_value'] - mean_rho) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)
```

```
    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rho} std={std_rho} support_fraction={support_fraction
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rho} std={std_rho} support_fraction={support_fraction
    else:
        first_failing_seed = next((r['seed'] for r in results if not r['conjecture_holds']), None)
        print(f"RESULT: FALSIFIED counterexample=\"Spearman rank correlation < 0.7\" first_failing
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a79f01e0.py", line 92, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a79f01e0.py", line 71, in run_trial
    dynkin = dynkin_diagram(n)
             ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a79f01e0.py", line 50, in dynkin_diagra
    raise ValueError("Unsupported number of variables for Dynkin diagram")
ValueError: Unsupported number of variables for Dynkin diagram
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed due to an error with unsupported number of variables for Dynkin diagram, which prevented the computation from producing any data. | next: Investigate and fix the error in the code that causes it to crash when encountering unsupported numbers of variables.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11564
2	preregistration	ollama_remote	glm4:latest	0	0	5377
3	novelty	ollama_remote	glm4:latest	0	0	4791
4	novelty	ollama_remote	glm4:latest	0	0	6004
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16147
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6598

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11280
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13150
9	judge	ollama_remote	glm4:latest	0	0	8272

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 83182 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/7cc4fa3cad8b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7cc4fa3cad8b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/7cc4fa3cad8b.tar.gz` (if generated)